

LABORATORY

LAB 16

BLDC Motor Control

Controlling a BLDC Motor through an ESC with EduFramework

S32K144 · MaaZEDU

EduFramework

FPT University

Contents

1	Introduction	2
1.1	Lab Overview	2
1.2	Objectives	2
2	Background	2
2.1	BLDC Motor and Electronic Speed Controller	2
2.2	Throttle Command and Arming	2
3	Hardware Setup	3
3.1	Required Hardware	3
3.2	Control Pin Mapping	3
3.3	ESC and Motor Connections	3
4	EduFramework API	4
4.1	ESC_Init()	4
4.2	ESC_Arm()	5
4.3	ESC_SetThrottle()	5
5	Lab Exercise	5
5.1	Requirements	5
5.2	Program	5
5.3	Verification	6
6	Extension	7
6.1	Extension Exercise	7
7	References	7

1 Introduction

1.1 Lab Overview

A BLDC (Brushless DC) motor uses electronic commutation instead of mechanical brushes and a commutator [1] [2]. In the system used in this lab, the Electronic Speed Controller (ESC) performs motor commutation and power control. The S32K144 does not directly control the motor phases; instead, it sends a control signal to the ESC.

This lab focuses on using the EduFramework ESC API to initialize the ESC, perform the safe startup process (arming), and change the throttle level, thereby controlling the operation of the BLDC motor.

1.2 Objectives

OBJECTIVES

The objectives of this laboratory are:

1. Describe the roles of the BLDC motor and Electronic Speed Controller in a motor control system.
2. Explain the meaning of a throttle command and the ESC arming process.
3. Connect the MaaZEDU board, ESC, external power supply, and BLDC motor according to the lab configuration.
4. Use `ESC_Init()`, `ESC_Arm()`, and `ESC_SetThrottle()` to control a BLDC motor.

2 Background

2.1 BLDC Motor and Electronic Speed Controller

A BLDC motor uses permanent magnets on the rotor and windings on the stator. Because mechanical brushes are not used, current in the windings must be electronically commutated to produce rotational torque [1] [2]. In this lab, the ESC performs this task. The S32K144 does not directly control the three `U`, `V`, and `W` terminals; the board only sends a control command to the ESC. The ESC receives a separate power supply through `B+` and `B-` and controls the three phases connected to the motor [3].

Note

The motor power supply must not be taken directly from a MaaZEDU GPIO. The external power supply must be suitable for the ESC and motor being used.

2.2 Throttle Command and Arming

EduFramework uses a throttle command from `0%` to `100%`. With the default configuration, `0%` corresponds to a pulse width of `1000 μs`, while `100%` corresponds to `2000 μs`; intermediate values are mapped linearly within this range [6].

The throttle percentage is the command value sent to the ESC, not the actual percentage of motor speed. The resulting RPM also depends on the motor, ESC, power supply, load, and operating conditions. If the actual speed is required, the system must use a measurement mechanism such as an encoder.

Before controlling the motor, the ESC must be placed in a safe startup state. In EduFramework, `ESC_Arm()` sends the minimum throttle and maintains this state for `3000 ms` before program execution continues [6].

3 Hardware Setup

3.1 Required Hardware

This lab uses the MaaZEDU Development Board with an S32K144 microcontroller from the S32K1xx family [4]. The ESC uses a separate power supply and drives the BLDC motor through its three phase outputs.

Hardware	Description
MaaZEDU Development Board	Development board based on the S32K144 microcontroller.
Electronic Speed Controller	Receives the RC PWM signal and controls the power stage of the BLDC motor.
BLDC Motor	Three-phase motor connected to the <code>U</code> , <code>V</code> , and <code>W</code> outputs of the ESC.
External DC Power Supply	External supply suitable for the ESC and motor being used.
Jumper wires	Connect GPIO2 and GND between the MaaZEDU board and the ESC.
USB Cable	Connect the board to the computer for programming.

Table 16.3.1. Hardware used in the lab

3.2 Control Pin Mapping

The lab uses `GPIO2` as the ESC control pin. On the MaaZEDU board, `GPIO2` is mapped to `PTD14` [7]. EduFramework identifies this Logical Pin as supporting the PWM function required by the ESC [6].

Component	Logical Pin	MCU Pin	Function
ESC PWM input	<code>GPIO2</code>	<code>PTD14</code>	ESC control signal

Table 16.3.2. ESC control signal mapping

3.3 ESC and Motor Connections

Connect `GPIO2` to the `PWM` input of the ESC and connect MaaZEDU GND to the control-side GND of the ESC. Connect the external power supply to `B+` and `B-`. Connect the ESC outputs `U`, `V`, and `W` directly to the three wires of the BLDC motor.

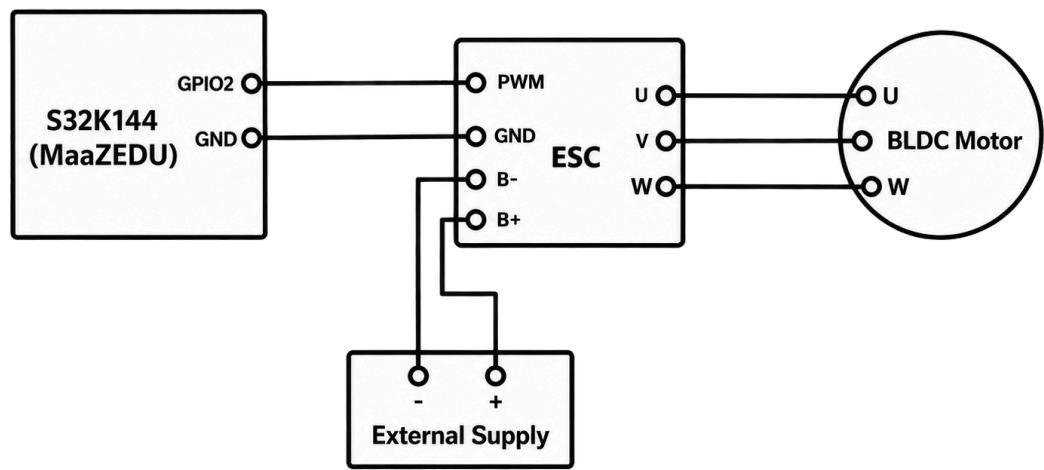


Figure 16.3.1. Connection diagram for the MaaZEDU board, ESC, and BLDC motor

Note

Disconnect the motor power supply before changing any wiring. Secure the motor before operation and begin verification at a low throttle level. Do not apply power until the supply voltage and current capability have been confirmed to be suitable for the ESC and motor.

4 EduFramework API

The main exercise uses three ESC Device APIs for initialization, arming, and setting the throttle [6] .

4.1 ESC_Init()

ESC_Init

Initializes the ESC on a Logical Pin that supports PWM.

Syntax

```
ESC_Init(pin);
```

Parameters

Parameter	Type / accepted value	Description
pin	Logical Pin	ESC control pin. This lab uses GPIO2 .

Return value

true if initialization succeeds; false if the selected pin is unsuitable or initialization fails.

4.2 ESC_Arm()

ESC_Arm

Sends the minimum throttle and performs the ESC arming interval.

Syntax

```
ESC_Arm();
```

Return value

No return value.

In EduFramework, the arming process lasts `3000 ms`, and the function is blocking during this interval.

4.3 ESC_SetThrottle()

ESC_SetThrottle

Sends a throttle command to the ESC as a percentage.

Syntax

```
ESC_SetThrottle(percent);
```

Parameters

Parameter	Type / accepted value	Description
percent	Throttle	Value from <code>0%</code> to <code>100%</code> ; values above <code>100%</code> are limited to <code>100%</code> .

Return value

No return value.

`ESC_SetThrottle()` controls the command value sent to the ESC; this API does not measure the actual motor RPM.

5 Lab Exercise

5.1 Requirements

Develop a program to control a BLDC motor through an ESC connected to `GPIO2`. After initialization and arming, the program changes the throttle according to the following sequence:

`20% → 40% → 60% → 40% → 20% → 0%`

Each throttle level is maintained for `3000 ms`. After returning to `0%`, the sequence repeats continuously. The objective is to observe the motor response as the throttle command increases and decreases; RPM measurement is not required in this lab.

5.2 Program

In `src/main.c`, implement the program that has been verified on the hardware:

```
#include "Arduino.h"
#include "esc.h"

int main(void)
{
    setup();

    ESC_Init(GPI02);
    ESC_Arm();

    while (1)
    {
        ESC_SetThrottle(20U);
        delay(3000U);

        ESC_SetThrottle(40U);
        delay(3000U);

        ESC_SetThrottle(60U);
        delay(3000U);

        ESC_SetThrottle(40U);
        delay(3000U);

        ESC_SetThrottle(20U);
        delay(3000U);

        ESC_SetThrottle(0U);
        delay(3000U);
    }

    return 0;
}
```

Listing 16.5.1. BLDC motor control program using a throttle sequence

`setup()` initializes EduFramework. `ESC_Init(GPI02)` prepares the control output, `ESC_Arm()` performs the arming process, and `ESC_SetThrottle()` then sends the specified throttle levels in sequence. `delay(3000U)` maintains each level for three seconds so that the response can be observed clearly [5].

The exercise does not use encoder feedback, so it verifies only the motor response to the command and does not determine the actual RPM.

5.3 Verification

Build and upload the program to the MaaZEDU board. Check `GPI02`, `GND`, `B+`, `B-`, and the three `U/V/W` wires before applying power to the ESC. After arming, observe the motor as the throttle increases from `20%` to `60%` and then decreases to `0%`.

EXPECTED RESULT

The ESC completes the arming process before the motor receives the throttle sequence. The motor responds to the `20%`, `40%`, and `60%` commands, then decreases through `40%` and `20%` before stopping at `0%`. The sequence repeats continuously. The result should demonstrate stable changes in response to the command; a linear relationship between throttle percentage and actual RPM is not required.

6 Extension

In addition to the three APIs used in the main exercise, EduFramework provides additional APIs for configuring and monitoring ESC commands [6] .

API	Purpose
<code>ESC_SetPulseRange()</code>	Changes the minimum and maximum pulse values used to map 0% to 100% throttle.
<code>ESC_SetMicroseconds()</code>	Directly sends a pulse width in microseconds within the configured range.
<code>ESC_GetThrottle()</code>	Reads the most recent throttle command stored in software.
<code>ESC_GetMicroseconds()</code>	Reads the most recent pulse-width command.
<code>ESC_IsInitialized()</code>	Checks whether the ESC Device has been initialized.
<code>ESC_End()</code>	Stops the output managed by the ESC Device and resets its internal state.

Table 16.6.1. Advanced ESC APIs

Note

`ESC_GetThrottle()` and `ESC_GetMicroseconds()` return only the commands that have been sent; they do not provide motor speed or physical feedback. `ESC_SetPulseRange()` should be changed only when the control range of the actual ESC has been identified.

6.1 Extension Exercise

Replace the fixed throttle sequence with a potentiometer connected to an Analog Input. Read the ADC value, scale it to a throttle command within an appropriate range, and then use `ESC_SetThrottle()` to control the motor.

Potentiometer → `analogRead()` → scale ADC → `ESC_SetThrottle()` → BLDC

7 References

- [1] James L. Kirtley Jr., *Course Notes 7: Permanent Magnet “Brushless DC” Motors*, MIT OpenCourseWare - 6.885 Electric Machines, 2013.
- [2] Deepak Mohanraj et al., *A Review of BLDC Motor: State of Art, Advanced Control Techniques, and Applications*, IEEE Access, Vol. 10, pp. 54833-54869, 2022.
- [3] Luc Hanneuse, *Output Devices - Brushless Motor and ESC*, Fab Academy - Sorbonne Lab, 2019.
- [4] NXP Semiconductors, *S32K1xx MCU Family - Data Sheet*, S32K1XX, Rev. 15, 2026.
- [5] Arduino, *Arduino Language Reference*, Arduino Documentation.
- [6] EduFramework, *ESC Device API*, EduFramework Source Code.
- [7] MaaZEDU, *MaaZEDU Development Board Guide*.